Toán tử lân cận là phép biến đổi một hành trình hiện tại thành một hành trình mới bằng cách thay đổi thứ tự các thành phố, trong khi vẫn đảm bảo mỗi thành phố xuất hiện đúng một lần.

Trong TSP, toán tử lân cận được sử dụng là Swap – hoán đổi vị trí của hai thành phố. Ví dụ, hành trình ban đầu là:

$$T = [0, 1, 2, 3, 4]$$

Sau đó, hoán đổi vị trí của thành phố 1 và thành phố 3, ta sẽ nhận được hành trình mới:

$$T' = [0, 3, 2, 1, 4]$$

→ Hành trình mới vẫn chứa đầy đủ năm thành phố, nhưng thứ tự di chuyển đã thay đổi. Vì vậy, các cạnh kết nối giữa các thành phố cũng có thể thay đổi, làm cho tổng quãng đường tăng hoặc giảm.

Và toán tử Swap được sử dụng trong TSP được tạo ra trong hàm swap_neighbors bằng cách:

1. Kiểm tra hành trình có chứa đủ các thành phố và không bị trùng lặp.
2. Liệt kê tất cả các cặp vị trí (i, j), với i < j.
3. Sao chép hành trình hiện tại.
4. Hoán đổi hai thành phố tại vị trí i và j.
5. Trả về hành trình mới sau khi hoán đổi.

Với n thành phố, số hành trình lân cận khác nhau có thể tạo ra bằng một lần hoán đổi là:

$$N = \frac{n}{2} = \frac{n(n-1)}{2}$$

→ Toán tử Swap tạo ra không gian tìm kiếm cục bộ để thuật toán có thể thử những cách sắp xếp thành phố khác nhau, từ đó tìm hành trình có tổng quãng đường ngắn hơn.

Tour Length là tổng khoảng cách của tất cả các chặng trong một hành trình, bao gồm cả chặng cuối cùng quay trở về thành phố xuất phát. Và đây cũng làm hàm mục tiêu của TSP, công thức được dùng là:

$$L(T) = \sum_{i=1}^{n-1} d_{\pi(i),\pi(i+1)} + d_{\pi(n),\pi_1}$$

Trong đó:

- $L(T)$: tổng độ dài hành trình.
- $n$: số lượng thành phố.
- $\pi(i)$: thành phố được ghé thăm ở vị trí thứ i.
- $d_{i,j}$: khoảng cách giữa thành phố I và thành phố j.
- $d_{\pi(n),\pi(1)}$ : khoảng cách từ thành phố cuối cùng quay lại thành phố đầu tiên.

Và tỷ lệ cải thiện nếu khi thuật toán tìm được hành trình mới có Tour Length tốt hơn khi đã giảm được so với hành trình cũ thì sẽ được tính bằng công thức:

$$I = \frac{L_{initial} - L_{final}}{L_{initial}} \times 100\%$$

Trong TSP, hàm tour_length sẽ thực hiện ba công việc chính:

- Khởi tạo tổng khoảng cách bằng 0.
- Duyệt qua hành trình và cộng khoảng cách giữa từng cặp thành phố liên tiếp.
- Cộng thêm khoảng cách từ thành phố cuối cùng quay về thành phố đầu tiên.

Sau đó, hàm trả về tổng độ dài của hành trình. Giá trị này được dùng để so sánh các hành trình và quyết định có nên chấp nhận một hành trình mới hay không. Đặc biệt, là TSP sẽ tạo ra 10 thành phố với tọa độ ngẫu nhiên và dùng chung một ma trận khoảng cách cho các thuật toán so sánh.

| Thuật toán | Nguyên lý hoạt động | Ưu điểm | Hạn chế |
|---|---|---|---|
| Steepest – ascent Hill Climbing | Thuật toán tạo toàn bộ các hành trình lân cận bằng phép Swap, tính Tour Length của từng hành trình và chọn hành trình có độ dài nhỏ nhất trong số những hành trình cải thiện được kết quả hiện tại.<br>Các bước thực hiện:<br>1. Bắt đầu từ hành trình ban đầu.<br>2. Sinh tất cả các hành trình lân cận.<br>3. Tính Tour Length của từng hành trình lân cận.<br>4. Chọn hành trình tốt nhất nếu nó có Tour Length nhỏ hơn hiện tại. | Có khả năng chọn bước cải thiện tốt nhất trong mỗi vòng lặp, giúp giảm Tour Length một cách | Phải đánh giá toàn bộ các hành trình lân cận ở mỗi bước. Thuật toán có thể dừng tại cực tiểu cục bộ, tức là không tìm |

| | | | |
|---|---|---|---|
| | 5. Lặp lại cho đến khi không còn hành trình lân cận nào tốt hơn. | có hệ thống. | thấy hành trình tốt hơn trong vùng lân cận hiện tại nhưng chưa chắc đó là nghiệm tối ưu toàn cục. |
| Steepest-ascent Hill Climbing with Random Restarts | Thuật toán chạy Steepest-ascent Hill Climbing nhiều lần với các hành trình bắt đầu khác nhau, sau đó giữ lại kết quả tốt nhất.<br>Và trong TSP, hàm steepest_ascent_hill_climbing_random_restarts() sử dụng hành trình ban đầu được cung cấp và tạo thêm 10 hành trình ngẫu nhiên. Tổng cộng có 11 lần chạy. | Tăng cơ hội tìm được nghiệm tốt hơn do khám phá nhiều vùng khác nhau của không gian tìm kiếm, giảm sự phụ thuộc vào một hành trình khởi tạo duy nhất. | Tốn thời gian và số lần tính hàm mục tiêu nhiều hơn so với chạy Hill Climbing một lần. Thuật toán vẫn không đảm bảo tìm được nghiệm tối ưu toàn cục. |
| Stochastic Hill Climbing | Thay vì chọn hành trình cải thiện tốt nhất, thuật toán thu thập tất cả các hành trình lân cận có Tour Length nhỏ hơn hiện tại rồi chọn ngẫu nhiên một hành trình trong số đó.<br>Quá trình tiếp tục cho đến khi không còn hành trình cải thiện nào. | Có tính ngẫu nhiên trong việc chọn bước tiếp theo, giúp quá trình tìm kiếm đa dạng hơn so với việc luôn chọn bước tốt nhất. | Không nhất thiết chọn được bước cải thiện lớn nhất. Kết quả có thể thay đổi theo hạt giống ngẫu nhiên và đường đi của quá trình tìm kiếm. |

| | | | |
|---|---|---|---|
| First-choice Hill Climbing | Thuật toán duyệt các hành trình Swap theo thứ tự ngẫu nhiên và chấp nhận ngay hành trình đầu tiên có Tour Length nhỏ hơn hành trình hiện tại.<br>Nếu tìm được hành trình cải thiện, thuật toán chuyển sang hành trình đó và bắt đầu tìm kiếm tiếp. Nếu đã thử hết các hàng xóm mà không có hành trình nào tốt hơn thì dừng lại. | Không cần đánh giá hết toàn bộ các hành trình lân cận sau khi đã tìm thấy một bước cải thiện. Nhờ vậy, thuật toán có thể giảm số lần đánh giá hàm mục tiêu và thời gian chạy. | Có thể bỏ qua hành trình cải thiện tốt hơn chỉ vì nó xuất hiện sau trong thứ tự duyệt. Kết quả phụ thuộc vào thứ tự ngẫu nhiên của các phép Swap. |
| Simulated Annealing | Simulated Annealing chọn ngẫu nhiên một cặp thành phố để hoán đổi, sau đó quyết định có chấp nhận hành trình mới hay không.<br>- Nếu hành trình mới ngắn hơn hoặc bằng hành trình hiện tại, thuật toán chấp nhận.<br>- Nếu hành trình mới dài hơn, thuật toán vẫn có thể chấp nhận với xác suất:<br>$$P = e^{(-\frac{\Delta L}{T})}$$<br>Trong đó:<br>$$\Delta L = L_{new} - L_{curent}$$<br>$T$: là nhiệt độ hiện tại. Nhiệt độ càng cao thì thuật toán càng có khả năng chấp nhận một bước làm tăng Tour Length. Khi nhiệt độ giảm, xác suất này cũng giảm. Và trong TSP, nhiệt độ ban đầu là 1.0, hệ số làm nguội là 0.995, nhiệt độ tối thiểu là 0.001 và giới hạn tối đa là 5.000 vòng lặp. | Có khả năng chấp nhận một số bước làm kết quả xấu đi, nhờ đó có thể thoát khỏi cực tiểu cục bộ mà các thuật toán chỉ chấp nhận bước cải thiện thường gặp phải. | Kết quả phụ thuộc vào các tham số nhiệt độ, tốc độ làm nguội và số vòng lặp. Thuật toán có thể tốn nhiều lần đánh giá hàm mục tiêu mà không cải thiện được nghiệm tốt nhất. |

| | | | |
|---|---|---|---|
| Genetic Algorithm – Thuật toán di truyền (Bonus) | Thuật toán duy truyền một quần thể gồm nhiều hành trình, sau đó sử dụng các cơ chế:<br>• **Tournament selection:** Chọn cá thể tốt nhất trong một nhóm được lấy ngẫu nhiên làm cá thể bố mẹ.<br>• **Order crossover:** Kết hợp thứ tự thành phố từ hai hành trình bố mẹ để tạo hành trình con hợp lệ.<br>• **Swap mutation:** Hoán đổi hai thành phố trong hành trình con nhằm tạo sự đa dạng.<br>• **Elitism:** Giữ lại hành trình tốt nhất qua các thế hệ để tránh làm mất nghiệm tốt nhất đã tìm được.<br>Trong TSP, kích thước quần thể là 50, số thế hệ là 200, xác suất lai ghép là 0.9 và xác suất đột biến là 0.1. | Khám phá nhiều hành trình cùng lúc và kết hợp các nghiệm tốt để tìm ra hành trình có Tour Length thấp hơn. | Cần đánh giá nhiều cá thể qua nhiều thế hệ, phụ thuộc vào kích thước quần thể và các tham số tiến hóa. Thuật toán cũng không đảm bảo nghiệm tối ưu toàn cục. |

Và hiệu quả các thuật toán trong TSP:

| Thuật toán | Tour Length cuối | Cải thiện | Số vòng lặp | Số lần đánh giá | Thời gian (ms) |
|---|---|---|---|---|---|
| Steepest – ascent | 2.76 | 3.11 | 7 | 360 | 2.04 |
| Random restarts | 2.76 | 3.11 | 50 | 2.745 | 42.29 |
| Stochastic | 2.76 | 3.11 | 21 | 990 | 7.61 |
| First – choice | 2.76 | 3.11 | 12 | 111 | 2.01 |
| Simulated annealing | 2.88 | 2.99 | 1.379 | 1.379 | 50.05 |

Ngoài ra, chương trình còn triển khai Genetic Algorithm ở phần Bonus. Thuật toán sử dụng chọn lọc, lai ghép thứ tự và đột biến hoán đổi để tiến hóa một quần thể hành trình. Sau 200 thế hệ, thuật toán tìm được Tour Length tốt nhất là 2.7865, giảm 3.0874 so với hành trình ban đầu, với 9.850 lần đánh giá hàm mục tiêu.